# CIA 2

# Design and Implementation of an End-to-End Cloud-Based CI/CD Pipeline

Project: RoboAdvisor Lite – Java Spring Boot application with CI/CD to Amazon EKS (AWS)

Submitted by

**Rahul Raghavendra R**

Register Number: 23011102070

B.Tech Computer Science and Engineering (IoT)

Shiv Nadar University, Chennai

GitHub Repository: https://github.com/rahulraghavendrar/robo-advisor-cicd

Date of Submission: 05 November 2026

# 1. Problem Definition

A software development team builds, tests, packages and deploys a Java web application manually. Every release requires a developer to run the Maven build on their own machine, execute the tests, build a Docker image, push it and then update the server. This process is slow, depends on the person doing it, is error-prone (for example, forgetting to run tests or deploying the wrong image) and is difficult to reproduce.

The aim of this assignment is to act as a DevOps engineer and automate this complete flow using a CI/CD pipeline on a public cloud. Whenever code is pushed to the Git repository, the pipeline must automatically build the application with Maven, run the unit tests, create a Docker image, publish it to a container registry and deploy it to a Kubernetes cluster hosted in the cloud, so that the change becomes visible in the browser without any manual steps.

For the application, I built a miniature version of the AI-Powered Robo-Advisor project I had earlier worked on, rewritten in Java with a simpler tech stack so that it fits the Maven based pipeline. The application is called RoboAdvisor Lite.

### 1.1 Application Overview

RoboAdvisor Lite is a Spring Boot web application. The user enters their age, risk appetite (Low / Medium / High), monthly investment amount and investment period. The application suggests an Equity / Debt / Gold allocation and projects the future value of a monthly SIP. The logic is rule based:

- Base equity percentage = 100 – age (common thumb rule).
- Risk appetite adjusts equity by –15% (Low), 0% (Medium) or +15% (High). Equity is kept between 10% and 85%.
- Gold is fixed at 10% and the remaining amount goes to debt.
- Expected yearly return is taken as 12% for equity, 7% for debt and 8% for gold, and the SIP future value is calculated monthly.

| Endpoint | Description |
|---|---|
| GET / | Web page (HTML form) to enter details and view the advice |
| GET /api/advice | Returns allocation and SIP projection as JSON, e.g. /api/advice?age=21&risk=HIGH&amount=5000&years=10 |
| GET /api/health | Returns {"status":"UP","version":"1.0.0"}; used by Kubernetes probes and the pipeline verify step |

# 2. Architecture

The overall architecture is shown in Figure 1. The developer pushes code to the main branch of the GitHub repository. This automatically triggers a GitHub Actions workflow with three jobs: (1) build and test using Maven, (2) build the Docker image and push it to Docker Hub, and (3) deploy the new image to an Amazon Elastic Kubernetes Service (EKS) cluster in the AWS Mumbai region (ap-south-1) using an Ansible playbook, followed by a verification step. The

application runs as two pods behind a LoadBalancer service, for which AWS creates an Elastic Load Balancer with a public DNS name so that the application can be opened in a browser.

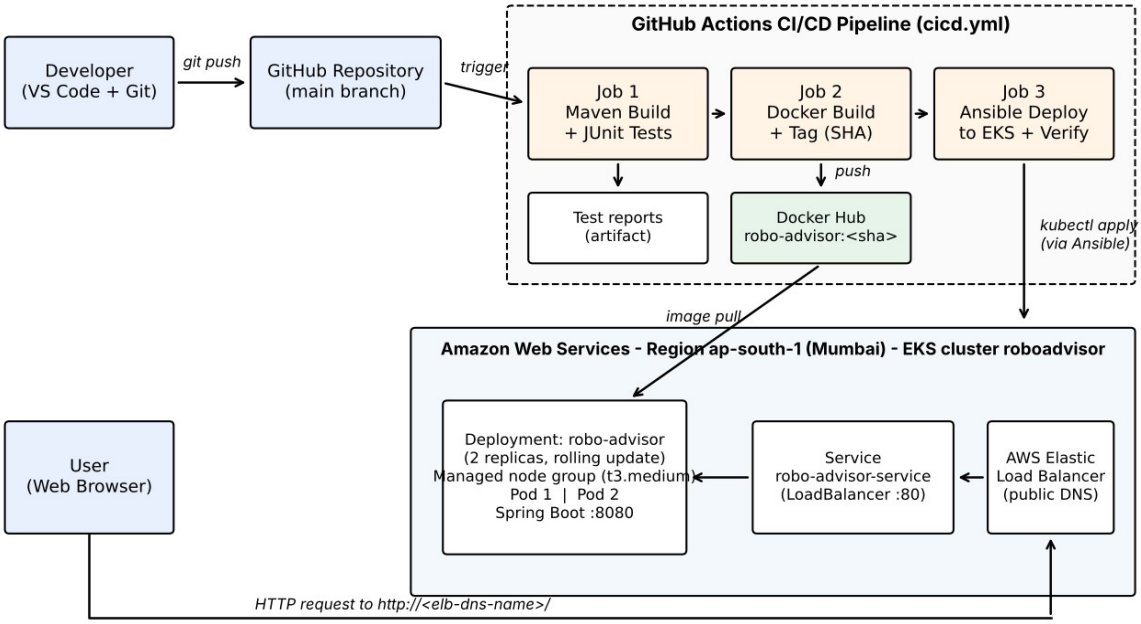



*Figure 1: Architecture of the CI/CD pipeline and deployment*

## 3. Tools Selected and Justification

| Area | Tool | Reason for choosing |
|---|---|---|
| Source code management | Git + GitHub | Industry standard, free public repositories, and it integrates directly with GitHub Actions so that a push can trigger the pipeline. |
| Build and test | Maven + JUnit 5 | Standard build tool for Java. pom.xml manages dependencies, and the Spring Boot plugin packages the app as a single executable jar. JUnit 5 and MockMvc are used for unit tests. |
| Application | Java 17, Spring Boot 3 | Spring Boot gives an embedded Tomcat server, so the jar runs directly in a container with no extra server setup. |
| CI/CD platform | GitHub Actions | No separate Jenkins server has to be installed and maintained. Free runners are available for public repositories and secrets are stored securely in the repository settings. |
| Containerization | Docker | Packages the app with its Java runtime so it runs the same on any machine. A small eclipse-temurin:17-jre-alpine base image is used. |
| Container registry | Docker Hub | Free public repository, easy login from GitHub Actions and AKS can pull public images without extra configuration. |
| Cloud provider | Amazon Web Services (AWS) | Most widely used cloud provider. EKS is a managed Kubernetes service, so AWS runs the control plane, and eksctl creates the whole cluster |

| Area | Tool | Reason for choosing |
|---|---|---|
| | | (VPC, control plane, node group) with one command. New accounts get free promotional credit. Microsoft Azure was tried first, but the Azure for Students subscription was disabled. |
| Orchestration | Kubernetes (Amazon EKS) | Manages replicas, restarts failed pods, does rolling updates without downtime and exposes the app using a LoadBalancer service backed by an AWS Elastic Load Balancer. |
| Configuration / deployment automation | Ansible | The deployment steps (set new image tag, apply manifests, wait for rollout, list pods) are written as an Ansible playbook so that the same deploy can be repeated from the pipeline or from a laptop. |

## 4. CI/CD Pipeline Stages

The pipeline is defined in .github/workflows/cicd.yml. It runs on every push to the main branch and can also be started manually (workflow_dispatch). The stages and their purpose are listed below.

| No. | Stage | Job / Step | Purpose |
|---|---|---|---|
| 1 | Checkout source code | actions/checkout@v4 | Gets the latest commit from the repository onto the runner. |
| 2 | Compile and package | mvn -B clean package -DskipTests | Downloads dependencies, compiles the code and creates target/robo-advisor.jar. |
| 3 | Run automated tests | mvn -B test | Runs all JUnit tests. If any test fails, the pipeline stops and nothing is deployed. Test reports are uploaded as an artifact. |
| 4 | Build and tag Docker image | docker build | Builds the image and tags it with the short commit SHA and with latest, so every commit has a unique image. |
| 5 | Push image to registry | docker push | Publishes both tags to Docker Hub after logging in with an access token. |
| 6 | Deploy / update on Kubernetes | aws eks update-kubeconfig + ansible-playbook ansible/deploy.yml | Connects kubectl to the EKS cluster using AWS credentials, writes the new image tag into the deployment manifest, runs kubectl apply and waits for the rolling update to finish. |
| 7 | Verify deployment | kubectl + curl | Shows deployment, pods, service and image, waits for the load balancer DNS name and calls /api/health to confirm the app is reachable from the internet. |

The jobs are connected using needs, so the Docker job runs only if build and tests pass, and the deploy job runs only if the image was pushed successfully. Docker Hub and AWS credentials are stored as GitHub repository secrets (DOCKERHUB_USERNAME,

DOCKERHUB_TOKEN, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY) and are never written in the code. The kubeconfig for EKS is generated inside the runner at runtime.

### 4.1 Dockerfile

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY target/robo-advisor.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Since the jar is already built by Maven in the pipeline, the Dockerfile only needs a Java runtime image and copies the jar into it. The container runs as a non-root user. The .dockerignore file excludes source code, Kubernetes and Ansible files so that the build context stays small.

### 4.2 Ansible Deployment Playbook (main tasks)

```
- name: Put the new image tag into the deployment manifest
  ansible.builtin.replace:
    path: "{{ manifest_dir }}/deployment.yaml"
    regexp: '^(\s*image:\s*).*$'
    replace: '\g<1>{{ image }}'

- name: Apply Kubernetes manifests (deployment + service)
  ansible.builtin.command: kubectl apply -f {{ manifest_dir }}

- name: Wait for the rolling update to finish
  ansible.builtin.command: kubectl rollout status deployment/{{ app_name }} --
timeout=180s
```

The inventory (ansible/inventory.ini) contains only localhost with a local connection, because the playbook runs on the GitHub Actions runner and talks to the cluster through kubectl. The image name with the commit SHA tag is passed from the pipeline using -e "image=...".

## 5. Cloud Resources and Deployment Approach

The application is deployed on Amazon Web Services in the Asia Pacific (Mumbai) region, ap-south-1. The resources used are:

| Resource | Details |
|---|---|
| EKS cluster | roboadvisor – managed Kubernetes control plane |
| Managed node group | workers – 1 x t3.medium EC2 instance (2 vCPU, 4 GB) |
| Networking | VPC, public/private subnets, internet gateway and NAT gateway created by eksctl through CloudFormation |
| Load balancer | AWS Elastic Load Balancer created automatically for the LoadBalancer service (public DNS name) |
| IAM user | github-actions – only eks:DescribeCluster permission in IAM, plus an EKS access entry for kubectl access |
| Container registry | Docker Hub public repository robo-advisor (tags: commit SHA and |

| Resource | Details |
|---|---|
| | latest) |

The cluster and the pipeline user are created from AWS CloudShell with the following commands:

```
eksctl create cluster --name roboadvisor --region ap-south-1 \
  --nodegroup-name workers --node-type t3.medium --nodes 1 --managed

aws iam create-user --user-name github-actions
aws iam put-user-policy --user-name github-actions --policy-name eks-describe \
  --policy-document '{..."Action":"eks:DescribeCluster"...}'
aws iam create-access-key --user-name github-actions
aws eks create-access-entry --cluster-name roboadvisor --principal-arn <user-
arn>
aws eks associate-access-policy --cluster-name roboadvisor --principal-arn
<user-arn> \
  --policy-arn
arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy --access-
scope type=cluster
```

In the pipeline, the deploy job logs in with the IAM user keys (aws-actions/configure-aws-credentials), runs aws eks update-kubeconfig to connect kubectl to the cluster and then runs the Ansible playbook.

**Status at the time of submission:** Microsoft Azure was the first choice, but the Azure for Students subscription was found Disabled. The deployment target was then moved to AWS. The AWS account created with the university e-mail is part of an AWS Organization whose service control policy explicitly denies EKS, so eksctl create cluster failed with an AccessDeniedException (Figure 2). The EKS cluster is therefore being created on a personal AWS account and will be shown in the live demonstration. No change to the pipeline, manifests or playbook is needed for this, only the AWS secrets.

The Kubernetes manifests are in the k8s folder:

- **deployment.yaml:** Deployment named robo-advisor with 2 replicas, RollingUpdate strategy (maxUnavailable 0, maxSurge 1) so that there is no downtime during updates, CPU/memory requests and limits, and readiness and liveness probes on /api/health.
- **service.yaml:** Service of type LoadBalancer that maps port 80 to container port 8080. On EKS, AWS creates an Elastic Load Balancer with a public DNS name for it.

[Insert screenshot: AWS CloudShell – eksctl create cluster denied by the organization service control policy on the student account]

*Figure 2: AWS CloudShell – eksctl create cluster denied by the organization service control policy on the student account*

[Insert screenshot: AWS console – EKS cluster roboadvisor (Active) and its node group [to be shown in live demonstration]]

*Figure 3: AWS console – EKS cluster roboadvisor (Active) and its node group [to be shown in live demonstration]*

[Insert screenshot: Docker Hub repository robo-advisor showing the image tags (commit SHA and latest)]

*Figure 4: Docker Hub repository robo-advisor showing the image tags (commit SHA and latest)*

# 6. Implementation Evidence and Test Results

## 6.1 Unit Tests

The project has two test classes with 12 test cases in total. AdvisorServiceTest tests the business logic directly, and AdvisorControllerTest uses Spring Boot MockMvc to test the REST endpoints.

| Test case | What is checked |
|---|---|
| allocationAlwaysAddsUpTo100 | Equity + Debt + Gold = 100% for all risk levels and ages from 18 to 95 |
| mediumRiskUsesHundredMinusAgeRule | Age 30, Medium risk gives 70% equity, 20% debt, 10% gold |
| highRiskGetsMoreEquityThanLowRisk | High risk investor gets more equity than low risk at the same age |
| youngerInvestorGetsMoreEquity | A 22 year old gets more equity than a 60 year old |
| equityStaysWithinLimits | Equity never goes above 85% or below 10% |
| projectedValueIsMoreThanInvested | Total invested is calculated correctly and the projected value is higher |
| sipWithZeroReturnEqualsAmountInvested | SIP formula returns exactly the invested amount when return is 0% |
| invalidInputsAreRejected | Invalid age, amount, years or risk level throws an error |
| healthEndpointReturnsUp | /api/health returns status UP and the version |
| adviceEndpointReturnsAllocation | /api/advice returns HTTP 200 with the correct allocation in JSON |
| badInputReturns400 | Invalid input returns HTTP 400 with an error message |
| homePageIsServed | The web page index.html is served |

[Insert screenshot: GitHub Actions – 'Run unit tests' step output showing Tests run and Failures: 0, BUILD SUCCESS]

*Figure 5: GitHub Actions – 'Run unit tests' step output showing Tests run and Failures: 0, BUILD SUCCESS*

## 6.2 Pipeline Execution

[Insert screenshot: GitHub Actions – pipeline run with Build and Test and Docker Build and Push jobs successful]

*Figure 6: GitHub Actions – pipeline run with Build and Test and Docker Build and Push jobs successful*

[Insert screenshot: Deploy job – Ansible playbook output with PLAY RECAP (ok, changed, failed=0) [to be shown in live demonstration]]

*Figure 7: Deploy job – Ansible playbook output with PLAY RECAP (ok, changed, failed=0) [to be shown in live demonstration]*

### 6.3 Kubernetes Deployment

[Insert screenshot: kubectl get deployment,pods,svc – READY 2/2, pods Running and service with ELB EXTERNAL-IP [to be shown in live demonstration]]

*Figure 8: kubectl get deployment,pods,svc – READY 2/2, pods Running and service with ELB EXTERNAL-IP [to be shown in live demonstration]*

[Insert screenshot: Application running in the browser at http://<elb-dns-name>/ showing the advice result and version 1.0.0 [to be shown in live demonstration]]

*Figure 9: Application running in the browser at http://<elb-dns-name>/ showing the advice result and version 1.0.0 [to be shown in live demonstration]*

## 7. Change and Redeployment Demonstration

To show that a code change reaches the deployed application automatically, the version shown in the footer of the page was changed from 1.0.0 to 2.0.0 in src/main/resources/application.properties. The steps followed were:

1. Edited app.version=2.0.0 in application.properties.
2. Committed and pushed the change: git commit -am "Update version to 2.0.0" followed by git push.
3. The push triggered the CI-CD Pipeline workflow automatically without any manual action.
4. A new Docker image tagged with the new commit SHA was built and pushed to Docker Hub.
5. Ansible updated the image in the EKS deployment and Kubernetes performed a rolling update, replacing the old pods with new ones.
6. Refreshing the browser at the same load balancer address shows Version: 2.0.0.

[Insert screenshot: Terminal – git commit and git push of the version change]

*Figure 10: Terminal – git commit and git push of the version change*

[Insert screenshot: GitHub Actions – new pipeline run triggered automatically by the push (commit message visible)]

*Figure 11: GitHub Actions – new pipeline run triggered automatically by the push (commit message visible)*

[Insert screenshot: Docker Hub – new image tag for the new commit]

*Figure 12: Docker Hub – new image tag for the new commit*

[Insert screenshot: kubectl get pods – new pods after the rolling update, with the new image tag [to be shown in live demonstration]]

*Figure 13: kubectl get pods – new pods after the rolling update, with the new image tag [to be shown in live demonstration]*

[Insert screenshot: Browser – application showing Version: 2.0.0 after redeployment [to be shown in live demonstration]]

*Figure 14: Browser – application showing Version: 2.0.0 after redeployment [to be shown in live demonstration]*

## 8. Challenges Encountered and Solutions

| Challenge | Solution |
|---|---|
| Pushing the same 'latest' tag again did not make Kubernetes restart the pods, so the change was not deployed. | Each image is tagged with the short commit SHA, and Ansible writes this tag into deployment.yaml before applying it. Since the image field changes every time, Kubernetes always does a rolling update. |
| Pods were restarted / marked not ready because Spring Boot takes some seconds to start. | Added initialDelaySeconds of 20 seconds for the readiness probe and 45 seconds for the liveness probe. |
| The Azure for Students subscription was Disabled, so Azure AKS could not be used. | Moved the deployment target to AWS EKS. Only the cluster connection step of the pipeline changed; the manifests and Ansible playbook stayed the same. |
| The student AWS account blocks EKS through an organization service control policy (AccessDeniedException for eks:DescribeClusterVersions). | Prepared the full cluster setup (eksctl, IAM user, EKS access entry). The cluster is created on a personal AWS account for the live demonstration, with no change to the pipeline. |
| Installing Ansible with pip on the Ubuntu runner can fail because system Python is externally managed. | Used actions/setup-python to get a separate Python and installed ansible-core into it. |
| On AWS the LoadBalancer service gets a DNS name instead of an IP, and a new load balancer takes a few minutes to become reachable. | The verify step reads the hostname from the service status and retries curl on /api/health for up to 6 minutes. |
| The pipeline needs access to EKS without storing passwords or kubeconfig files in the code. | A separate IAM user with only eks:DescribeCluster permission and an EKS access entry is used. Its keys are GitHub secrets, and the kubeconfig is generated at runtime with aws eks update-kubeconfig. |

## 9. Security Considerations

- All credentials (Docker Hub username and token, AWS access keys) are stored as encrypted GitHub Actions secrets and are masked in the pipeline logs. Nothing sensitive is committed to the repository; kubeconfig files are also listed in .gitignore.
- A Docker Hub access token is used instead of the account password, and a separate IAM user with minimum IAM permissions is used for the pipeline instead of the root account. Both can be revoked at any time.
- The container runs as a non-root user and uses a small JRE-only Alpine base image, which reduces the attack surface.
- CPU and memory limits are set on the pods so that one container cannot use all the node resources.
- Tests act as a quality gate – the image is never built or deployed if a test fails.

## 10. Limitations

- At the time of submission the EKS deploy stage could not run on the student AWS account because EKS is blocked by the organization policy; the deployment is demonstrated live on a personal AWS account.
- The application is exposed over plain HTTP through the load balancer. In production, HTTPS (ACM certificate with an Ingress / ALB) and a domain name should be used.
- Long-lived IAM access keys are used. GitHub OIDC with an IAM role would avoid storing keys at all.
- EKS is charged per hour (control plane, node, NAT gateway, load balancer), so the cluster is deleted after evaluation with eksctl delete cluster.
- Only unit tests are run. Integration tests, code quality checks (e.g. SonarQube) and image vulnerability scanning (e.g. Trivy) can be added as extra stages.
- A single node cluster is used, so it is not highly available. There is only one environment (no separate staging and production).

## 11. Conclusion

A complete CI/CD pipeline was designed and implemented for a Java Spring Boot application using Git, Maven, Docker, GitHub Actions, Ansible and Kubernetes on Amazon EKS. Every push to the main branch is automatically built, tested, containerized, published to Docker Hub and rolled out to the EKS cluster without downtime, and the pipeline verifies that the application is reachable through the AWS load balancer. The change demonstration shows that a simple version change goes from a git push to the live application without any manual steps, which solves the original problem of slow and error-prone manual releases.

## 12. Source Code Repository

**GitHub:** https://github.com/rahulraghavendrar/robo-advisor-cicd

The repository contains the Java source code, pom.xml, unit tests, Dockerfile, .dockerignore, the GitHub Actions workflow (.github/workflows/cicd.yml), Kubernetes manifests (k8s/), the Ansible playbook and inventory (ansible/) and a README with setup steps.